

AI-Driven API Security Monitoring, Intelligent Threat Detection, and Automated Incident Response Platform Using Large Language Models

T. Aashritha Reddy, N. Sai Pragathi, Seera Indu

Department of Computer Science and Engineering, K L (Deemed to be) University

Draft status note (delete before submission). Text in [square brackets] must be filled by the team. This draft contains **no experimental numbers of our own**. Sections XI–XIII are written so you can insert real results as you obtain them. Section IV describes ALERT based only on its published abstract; please check it against the full text before submission (see Section IV).

ABSTRACT

Web APIs now carry most of the business logic and data exchanged by modern applications, yet many API attacks are hard to detect because each individual request looks legitimate. Enumeration of resource identifiers, broken-object-level authorization abuse, credential attacks, and slow data exfiltration are visible mainly in the *sequence* of requests made by a session, user, or IP address. The recent ALERT framework (Sooksomstarn, Wu, and Kanai, IEEE ICCE 2026) demonstrated a lightweight closed-loop SOC agent for web and API security that combines CVE severity prediction, runtime log anomaly detection, LLM-assisted semantics, cost-sensitive decision making, and ALLOW/MONITOR/BLOCK responses with a human in the loop. Building on that work, this paper proposes an API-focused, behavior-aware security monitoring and incident response framework that combines machine-learning-based anomaly detection with LLM-based contextual reasoning, explainability, risk assessment, and controlled automated response. Our two central extensions are (1) API-specific behavioral anomaly detection over session-, user-, and IP-level request sequences, and (2) API-specific graded incident response (monitor, rate limiting, temporary IP blocking, session/token revocation, endpoint restriction, administrator alert, and full blocking) selected by a transparent risk score under a policy engine that the LLM cannot override. The LLM is used for contextual reasoning, incident summarization, and recommendation, not as the detector. We describe the architecture, feature design, risk model, safeguards against prompt injection and unsafe actions, a Spring Boot/React/PostgreSQL implementation plan, and a full experimental methodology including an ALERT-style baseline. Empirical results are reported as future/in-progress work.

Keywords: API security; anomaly detection; large language models; incident response; risk scoring

I. INTRODUCTION

Application programming interfaces (APIs) are the main interface between clients, services, and data stores. The OWASP API Security Top 10 (2023 edition) lists authorization failures such as Broken Object Level Authorization (BOLA) and Broken Authentication among the most serious API risks [11]. Many of these attacks are *behavioral*: no single request contains a malicious payload, but the way requests are chained reveals intent. For example, a client that requests `/api/user/101`, `/api/user/102`, `/api/user/103`,

... `/api/user/5000` in quick succession is performing identifier enumeration, even though each request is syntactically valid and may return HTTP 200.

Recent research supports this observation. Yang et al. note that broken access control violations are often composed of multiple correlated requests that look normal in isolation [2], and Dong et al. describe "behavior-level" attacks that appear benign in individual flows but reveal malicious purpose across multiple flows [4]. At the same time, security operations centers (SOCs) face alert overload, which has motivated LLM-based alert investigation and triage [5], [8].

The base paper for this work, ALERT [1], is a 2026 IEEE ICCE contribution that presents a lightweight, CPU-only, closed-loop SOC agent with human-in-the-loop operation for web and API security. Its published abstract reports the combination of CVE severity prediction, runtime log anomaly detection, LLM-assisted semantics, cost-sensitive decisions, online policy adaptation, and ALLOW/MONITOR/BLOCK responses, evaluated on 10,000 synthetic Apache-style logs [1]. We explicitly acknowledge that ALERT already contains LLM assistance and automated (closed-loop) response.

Our contribution. This paper does not replace ALERT; it proposes an extension specialized for API traffic. We present:

1. **API-specific behavioral anomaly detection** using request-, window-, and sequence-level features (endpoint template, method, identity, frequency, status codes, latency, payload size, endpoint sensitivity, repeated resource-ID access, call sequences, session behavior).
2. **API-specific intelligent incident response** with a graded action set (monitor, rate limit, temporary IP block, session/token revocation, endpoint restriction, administrator alert, full block) chosen from an API-specific risk score under a policy engine.
3. **A constrained LLM role:** contextual reasoning, event correlation, human-readable explanation, incident summarization, and response recommendation, with safeguards (structured prompts, output validation, restricted actions, data minimization, human approval, and prompt-injection defenses).
4. **A Spring Boot / React / PostgreSQL implementation blueprint and an experimental methodology** with ablations and an ALERT-style baseline.

Formally, our contribution is: *an API-focused, behavior-aware security monitoring and incident response framework that*

combines machine-learning-based anomaly detection with LLM-based contextual reasoning, explainability, risk assessment, and controlled automated response. We do not claim it is the first system of its kind; we have not performed a systematic literature review.

The rest of the paper is organized per the section list above.

II. BACKGROUND AND MOTIVATION

A. API attack surface. REST APIs expose resources through predictable URL patterns and identifiers, which makes them attractive for enumeration and authorization abuse. Typical behaviors include credential stuffing and brute force against login endpoints, ID walking against `/resource/{id}`, abuse of expensive endpoints, session/token replay from multiple IPs, and gradual bulk extraction of records.

B. Why per-request analysis is insufficient. Signature- and rule-based filters inspect a request or a small number of fields. Behavioral attacks are defined by *relations between requests*: identifier deltas, repetition, call order, status-code mix, and deviation from the caller's own history. BacAlarm's motivating example shows that two requests can be nearly indistinguishable in syntax and status yet differ in cross-request context such as token switching and identity inconsistency [2].

C. Why a graded response matters. Blocking every suspicious source is costly: false positives disrupt legitimate users and integrations. Practical systems apply progressively stronger controls. Yang et al. report that in their industrial collaborations, organizations often begin with warnings or temporary rate limits and escalate to blocking only if suspicious behavior persists [2]. This motivates a response ladder rather than a binary decision.

D. Role of LLMs. LLMs are good at reading heterogeneous context and producing explanations, but they are stochastic and vulnerable to prompt injection when they process attacker-controlled text. Therefore they should support analysts and policy, not replace quantitative detectors or act with unrestricted authority.

III. RELATED WORK

This section reviews recent (2025–2026) work relevant to API security, anomaly detection, LLM-based analysis, and SOC automation. For each work we state what was proposed and built, the evaluation, the contribution, the remaining opportunity, and how our work relates. Descriptions are based on the abstracts and accessible text of the cited papers; please re-verify details against the originals before submission. Where a detail was not available to us, we say so rather than guess.

A. API-level behavioral and access-control anomaly detection
BacAlarm (Yang et al., 2025) [2].

Proposed/developed: A two-part approach for detecting Broken Access Control violations: an LLM-and-RAG-based generator that simulates labeled composite API traffic from an API knowledge base mined from logs, and a detector that uses static features plus sequential (API-syntax and entropy-based) features in a hybrid CatBoost-plus-MLP ensemble.

Evaluation: Eight API sets from two open-source applications (Humhub and Memos) with real CVE-linked vulnerabilities;

the authors report average F1 and MCC improvements of 21.2% and 24.1% over the strongest baseline per set.

Contribution: Shows that cross-request sequential features are decisive (ablations show large drops without them) and that LLM-simulated traffic can address scarcity of labeled composite traffic.

Limitation/opportunity: The authors acknowledge a gap between models trained on simulated data and on real data, that data came from CVE-based reproductions rather than live traffic, and that the approach is best applied to security-critical APIs. It focuses on detection, not on a complete response, explanation, and dashboard pipeline.

Relation to ours: BacAlarm is the closest technical precedent for our sequence-aware features and for using ensembles; we add risk scoring, a graded response policy, LLM explanation, and an operator dashboard. Its discussion of graded interventions supports our response ladder.

BLADE (Dong et al., 2025) [4].

Proposed/developed: An unsupervised traffic anomaly detection system for web services that targets both flow-level and behavior-level attacks: a flow autoencoder yields reconstruction-based scores, unsupervised clustering assigns pseudo-operation labels, and a one-class classifier over aggregated multi-flow triplets characterizes legitimate operation patterns.

Evaluation: A custom dataset and CIC-IDS2017, with reported F1 of 0.9732 and 0.9801 respectively.

Contribution: Demonstrates that multi-flow behavioral aggregation catches attacks that single-flow analysis misses.

Limitation/opportunity: It operates on network traffic patterns; it does not, as far as we could see, reason over API semantics (endpoint sensitivity, resource IDs, user/session identity) or provide response and explanation layers.

Relation to ours: Supports our use of one-class/unsupervised scores (Isolation Forest, One-Class SVM) over aggregated behavior; we operate at the API-log level with application identity and add response and explanation.

MINES (Zhang et al., ICSE 2026) [3].

Proposed/developed: Explainable anomaly detection for web APIs by inferring API invariants at the schema level. It converts API signatures into table schemas, infers database constraints, uses an LLM to propose relationships between table structures, accepts or rejects LLM-generated invariants using normal logs, and translates invariants into executable Python checks.

Evaluation: Web-tamper attacks on TrainTicket, NiceFish, Gitea, Mastodon, and NextCloud, compared with LogRobust, LogFormer, and WebNorm (see the paper for quantitative results).

Contribution: Invariant-based, explainable detection that can flag behavior invisible in raw logs, and a disciplined use of the LLM whose proposals are validated against normal data.

Limitation/opportunity: In our reading, it depends on access to API signatures and database schema/state, which may not be

available in every deployment, and it targets detection rather than graded response.

Relation to ours: Its pattern of "LLM proposes, data validates" parallels our "LLM recommends, policy validates." Invariants could later be added as features in our detector.

Aharon et al. (2025) [15].

Proposed/developed: A few-shot, classification-by-retrieval framework for API injection anomaly detection. Their arXiv version builds a language model from normal API traffic and classifies by retrieval.

Evaluation/limitation: We did not review the evaluation details; we cite it as an example of payload-level API anomaly detection.

Relation to ours: It addresses injection-related behavior at the payload level, complementing our behavioral focus; a payload-level score could be an additional feature in our pipeline.

Datasets. FUR-API [13] and the Kaggle API access-behavior anomaly dataset [14] are candidate public resources; their suitability for our feature set must be checked (see Section XII).

B. LLM-based security analysis, SOC automation, and API vulnerability detection

Agentic investigation of security alerts (Eilertsen, Mavroedis, and Grov, 2025/2026) [5].

Proposed/developed: An experimental agentic workflow in which LLMs, given predefined queries and constrained tools (structured SQL over Suricata logs and grep-style search), automate the first stages of alert investigation: they choose queries, extract raw evidence, and deliver a verdict.

Evaluation: Compared against a deliberately simple baseline workflow on simple attacks such as brute force; the authors report a clear improvement and relatively consistent outputs with prompt sanitation, while noting run-to-run LLM variation and narrow experimental scope.

Contribution: Practical guardrails (time-bounded, parameterized queries; constrained tools) for LLM-driven investigation.

Limitation/opportunity: Network-IDS alerts and simple attack types; no API-behavior model or graded response.

Relation to ours: We adopt the same principle: a constrained LLM with a fixed evidence schema. Unlike theirs, our LLM receives pre-computed, minimized API evidence and cannot execute queries.

Agentic security survey [8]. This survey covers defensive security agents; it summarizes Turcotte et al. (2025), an AI-based SOC framework using density-based clustering and isolation forests to prioritize mixed SIEM alerts, and Tellache et al. (2025), a RAG-based agent that enriches SIEM alerts with threat intelligence to produce mitigation plans for APT-related alerts. We cite these two only as described in the survey and have not read the primary papers. *Opportunity:* API-specific behavior and graded API actions are not the focus of these SOC-level works. *Relation:* They support our use of isolation-forest-style scoring and LLM-based enrichment.

DAST-AI (2026) [6].

Proposed/developed: A hybrid dynamic testing framework that uses an LLM as a post-execution semantic oracle to judge whether HTTP responses improperly expose sensitive data, aiming to reduce false positives for authorization flaws (BOLA, missing authentication).

Evaluation: Juice Shop and VAmPI; the authors report that an authenticated reference comparison reaches F1 of 0.28 on Juice Shop and 0.857 on VAmPI, arguing that semantic reasoning about response content is the differentiator.

Contribution: Shows LLMs can add semantic judgment where rule-based scanners cannot.

Limitation/opportunity: It is a pre-deployment testing approach, not runtime behavioral monitoring or response. [Add authors/venue details from the source.]

Relation to ours: Supports the value of LLM semantic reasoning in API security; we apply LLM reasoning to runtime incidents instead of test results.

BOLA taxonomy from bug-bounty disclosures (2026) [7].

Proposed/developed: An empirical taxonomy of BOLA from publicly disclosed bug bounty reports, using LLM-assisted, human-adjudicated classification.

Evaluation: 200 disclosures sampled, 107 fully classified, of which 84 (78.5%) were confirmed in-scope BOLA, according to the abstract we accessed.

Contribution: Grounds BOLA and IDOR patterns in real disclosures. [Add authors/venue from the source.]

Limitation/opportunity: It is an analysis of vulnerabilities, not a detector.

Relation to ours: Motivates enumeration and repeated-resource-ID features and the threat categories in Section VIII.

C. Explainable AI

Explainability appears in MINES (explainable invariants) [3] and in BacAlarm (SHAP analysis of sequential features) [2]. Earlier work on explainable API anomaly detection (ACM ICMLC 2024) [9] and on combining a Random Forest detector with SHAP/LIME and a conversational LLM (HuntGPT, 2023) [10] pre-date 2025 but are the closest precedent for pairing a classical detector with LLM-generated explanation. *Opportunity:* Explanation faithfulness (whether the text matches actual evidence) is rarely measured; we propose to measure it (Section XII).

D. Summary of the gap

Recent work covers behavioral API detection [2], [4], schema/invariant reasoning [3], LLM-based alert investigation [5], and LLM semantic testing [6], and ALERT covers closed-loop response for web/API security [1]. In the works we reviewed, we did not find one that combines API-session behavioral detection, a transparent API-specific risk score, a graded API response ladder with policy constraints, and LLM explanation in one evaluated pipeline. This is an observation from a limited, non-systematic review and not a novelty claim.

IV. BASE PAPER: ALERT

Reference. T. Sooksomstarn, Q. Wu, and A. Kanai, "ALERT: A Closed-Loop SOC Agent for Web and API Security with

CVE-Based Intelligence and Adaptive Anomaly Detection," 2026 IEEE International Conference on Consumer Electronics (ICCE), 2026, DOI 10.1109/ICCE67443.2026.11449910 [1].

What ALERT contributes (per its published abstract). ALERT (Agentic Learning for Event-driven Response and Threat Detection) is a lightweight, CPU-only, closed-loop SOC agent with human-in-the-loop. It integrates CVE severity prediction, runtime log anomaly detection, LLM-assisted semantics, vulnerability feeds, runtime logs, analyst annotations, cost-sensitive decision making, and online policy adaptation, and produces ALLOW, MONITOR, or BLOCK responses. On a synthetic dataset of 10,000 Apache-style logs it reports precision 0.991, recall 0.983, F1 0.987, 100% detection of critical threats, and false positives below 0.5% [1].

Acknowledgment. ALERT already uses LLM assistance and already performs automated closed-loop response. We therefore do not position our work as adding an LLM or automation to ALERT.

Scope of our reading. Our description relies on the abstract. We have not verified from the full text how ALERT represents API-specific features, whether it models multi-request sessions, how its LLM is used, or whether it includes a dashboard. Wherever the abstract is silent, we make no claim either way. **Action for the team:** read the full paper and adjust this section and the comparison table accordingly.

How we use ALERT. ALERT provides (i) the closed-loop, human-in-the-loop architecture pattern, (ii) the idea of cost-sensitive decisions, and (iii) a natural baseline. We build on these and focus on the two extensions below.

V. RESEARCH GAP AND MOTIVATION FOR THE PROPOSED SYSTEM

We frame the following as **research opportunities** rather than deficiencies of ALERT.

Opportunity 1: API-specific behavioral modeling. APIs expose structure that generic runtime logs only partially capture: endpoint templates, resource identifiers, method semantics, authenticated identity, and per-endpoint sensitivity. Related work shows that sequences over such structure expose attacks invisible per request [2], [4]. Our extension makes this behavioral view the core of detection.

Opportunity 2: API-specific graded response. ALERT's three-level response is a sound general design. API environments offer finer, cheaper controls, such as rate limiting one session, revoking a JWT, or restricting one endpoint, which lower the cost of a wrong decision compared with a full block. Our extension defines these actions and ties them to risk and threat type.

Opportunity 3: Evidence-grounded explanation. SOC analysts need to know what happened, why, and what evidence supports a decision. We use the LLM specifically to produce structured, evidence-linked explanations and incident summaries, and we propose to measure their faithfulness.

Opportunity 4: Safe LLM integration. Log content is attacker-controlled, so an LLM that reads it is exposed to prompt injection. We treat safe integration as a design requirement.

VI. PROPOSED SYSTEM

The proposed system is an API-focused security monitoring and response platform with the following properties:

- **Behavior-first detection.** Features are computed per request and over sliding windows per session, user, and IP; sequence features capture call order and identifier progression.
- **Quantitative detection, contextual reasoning.** ML models produce the threat class, confidence, and anomaly score. The LLM does not detect; it explains, correlates, summarizes, and recommends.
- **Transparent risk score.** A weighted score combines model confidence, anomaly score, threat severity, request frequency, endpoint sensitivity, and deviation from historical behavior.
- **Policy-controlled response.** A deterministic policy engine maps risk and threat type to allowed actions. The LLM's recommendation is advisory and is validated against policy.
- **Human oversight.** High-impact actions require administrator approval; all actions are audited and reversible where possible.
- **Operational visibility.** A dashboard exposes traffic statistics, incidents, explanations, and actions.

Threat categories. The system is designed to label: (1) normal API behavior; (2) authentication attacks; (3) brute-force attacks; (4) authorization violations; (5) API enumeration; (6) injection-related behavior; (7) rate abuse; (8) suspicious session behavior; (9) abnormal API sequences; (10) possible data-exfiltration behavior. **These are design targets. Validation status for each category will be stated in Section XIII once experiments are run; we do not claim all have been validated.**

ALERT comparison.

Feature	ALERT 2026 [1]	Our Proposed System
Web/API security	Yes (per title and abstract)	Yes, API-focused
CVE intelligence	Yes (CVE severity prediction, vulnerability feeds)	Not a core component in the current scope; we focus on runtime behavior. Optional future integration: mapping known CVEs/dependencies to affected endpoints and using them as a risk feature
Log anomaly detection	Yes (runtime log anomaly detection)	Our proposed extension focuses on API behavioral anomaly detection over sessions, users, IPs, and call sequences
LLM assistance	Yes (LLM-assisted semantics)	LLM contextual reasoning, event correlation, explanation, incident summary, response recommendation, with restricted authority
Input	Vulnerability feeds, runtime logs, analyst annotations	API request/response metadata and security-event data (endpoint, method, IP, user/session, status, latency, size, resource IDs), plus analyst feedback
Evaluation data	10,000 synthetic Apache-style logs	API-focused test environment and/or dataset [to be finalized, see Section XII]

Feature	ALERT 2026 [1]	Our Proposed System
Response	ALLOW / MONITOR / BLOCK	Monitor / rate-limit / temporary IP block / session-token revocation / endpoint restriction / administrator alert / full block
API behavioral analysis	Not described in the abstract; not assessed here	Core focus
Risk scoring	Cost-sensitive decision framework	API-specific risk score combining ML, anomaly, severity, frequency, sensitivity, and history
Dashboard	Not described in the abstract; not assessed here	API security monitoring dashboard
Human-in-the-loop	Yes	Controlled analyst oversight with approval for high-impact actions
Explainability	LLM-assisted semantics	LLM-generated API incident explanations with evidence references

This table describes what each system emphasizes. It is not evidence that ALERT lacks any capability the abstract does not mention.

VII. SYSTEM ARCHITECTURE

The pipeline is:

API Traffic → **API Log Collection** → **Preprocessing** → **Feature Extraction** → **Behavioral Anomaly Detection** → **Threat Classification** → **Risk Scoring** → **LLM-Based Security Reasoning** → **Automated API Response** → **Security Dashboard**

[Fig. 1 flow] API Traffic → API Log Collection → Preprocessing → Feature Extraction → Behavioral Anomaly Detection → Threat Classification → Risk Scoring → LLM-Based Security Reasoning → Automated API Response → Security Dashboard (feedback loops: response → feature extraction; analyst feedback → classification). Redraw as a block diagram before submission.

Fig. 1. Proposed architecture.

A. API Traffic. Requests and responses pass through the protected Spring Boot services (and, optionally, a gateway).

B. API Log Collection. A servlet filter records per-request metadata: timestamp, endpoint, method, source IP, user/session ID (from the JWT subject and session identifier), status code, response time, request size, and response size. Bodies and secrets are not stored by default. Events are written asynchronously to PostgreSQL to limit overhead.

C. Preprocessing. Normalize URLs into endpoint templates (/api/user/{id}), extract numeric or UUID resource identifiers into a separate field, pseudonymize user identifiers where possible, discard malformed records, and attach endpoint sensitivity from a configuration table.

D. Feature Extraction. Compute request-level, window-level, and sequence-level features (Section VIII).

E. Behavioral Anomaly Detection. Unsupervised/semi-supervised models (Isolation Forest, One-Class SVM) trained on normal windows yield an anomaly score.

F. Threat Classification. Supervised models (Random Forest, XGBoost, Gradient Boosting, Logistic Regression) assign one of the threat categories with a calibrated confidence.

G. Risk Scoring. Combines model outputs and context into a 0–100 score and a risk band (Section X).

H. LLM-Based Security Reasoning. Receives a minimized, structured evidence bundle and returns a validated JSON explanation and recommendation (Section IX).

I. Automated API Response. A policy engine selects and executes allowed actions through Spring Security components and rate-limit/deny-list stores, with human approval where required (Section X).

J. Security Dashboard. Shows traffic, incidents, and actions and captures analyst feedback for retraining.

VIII. API BEHAVIORAL ANOMALY DETECTION

A. Why sequences matter

A single request GET /api/user/103 returning 200 is normal. A sequence of 4,900 such requests with consecutive identifiers from one session within minutes is not. The detector therefore scores *entities over windows* rather than isolated requests.

B. Feature design

Request-level: endpoint template, HTTP method, status code, response time, request size, response size, endpoint sensitivity level, authenticated/unauthenticated flag.

Window-level (per session, user, and IP; e.g., sliding windows of 1 and 5 minutes, sizes to be tuned):

- request count and rate; burstiness (peak-to-mean rate);
- number of distinct endpoint templates and distinct source IPs per session/token;
- failed-authentication count; ratio of 401/403 and of 4xx/5xx responses; status-code entropy;
- mean and variance of response time and payload size; total bytes returned;
- proportion of requests to high-sensitivity endpoints.

Resource-ID features (for enumeration/BOLA-style behavior): for an endpoint template with an identifier parameter, within a window let I be the ordered list of accessed identifiers.

- D_{id} = number of distinct identifiers;
- run_{len} = length of the longest run where consecutive identifier differences are small (e.g., $\leq \delta$);
- seq_{frac} = fraction of consecutive pairs with difference equal to +1 or -1;
- $cross_{owner}$ = number of identifiers not owned by the caller, when ownership can be looked up (authorization-violation signal).

Sequence-level: endpoint transition entropy; rarity of endpoint bigrams relative to a normal transition matrix estimated on benign traffic (analogous in spirit to the transition-entropy and next-event likelihood features used in BacAlarm [2]); repeated-cycle indicator (e.g., login → resource → login loops).

Historical baseline features: deviation of the current window from the entity's own rolling baseline (z-score or percentile of rate, endpoint mix, and time-of-day).

C. Models

We plan to compare:

Model	Role	Note
Isolation Forest	Unsupervised anomaly score	Needs no attack labels
One-Class SVM	Unsupervised/semi-supervised anomaly score	Sensitive to kernel and scaling
Logistic Regression	Interpretable supervised baseline	Useful for calibration and weights
Random Forest	Supervised multi-class classifier	Robust on tabular data
Gradient Boosting	Supervised classifier	
XGBoost	Supervised classifier	

We do **not** assume in advance that any model is superior. The final model (or an ensemble) will be chosen by the experimental comparison in Section XII, using time-aware validation, per-class metrics, false-positive rate, and latency.

D. Training data strategy

Given the scarcity of labeled API attack traffic, the plan combines (i) benign traffic from the testbed, (ii) scripted attack traffic generated against the testbed (Section XII), and (iii) optionally public datasets [13], [14]. LLM-simulated traffic, as in BacAlarm [2], is a possible future data source but is not assumed here.

IX. LLM-BASED THREAT REASONING AND EXPLANATION

A. Role

The LLM is a reasoning and communication layer. It **does not** decide whether traffic is malicious (the ML pipeline does), and it **cannot** directly execute actions. It supports six functions:

1. **Understanding the detected event.** Interpret the structured evidence (features, scores, class, endpoints).
2. **Correlating multiple API events.** Link related events (same IP across several sessions, same token from multiple IPs, enumeration followed by data retrieval) into one incident.
3. **Natural-language explanation.** Answer: What happened? Why is it suspicious? What threat type is likely? How severe is it? What evidence supports the decision? What response is recommended?
4. **Incident summarization.** Produce concise summaries for the dashboard and reports.
5. **Response recommendation.** Suggest an action from the allowed set with a justification.
6. **Analyst support.** Answer follow-up questions grounded in the incident's stored evidence.

B. Structured input and output

The prompt has a fixed system section (role, rules, allowed actions), and a clearly delimited data section holding a JSON evidence bundle. Example output schema:

```
{
  "incident_id": "...",
  "what_happened": "...",
  "why_suspicious": "...",
  "likely_threat_type": "api_enumeration",
  "severity": "high",

```

```
"evidence": [{"field": "seq_frac", "value": 0.98},
{"field": "D_id", "value": 4900}],
"recommended_action": "revoke_session",
"confidence_note": "...",
"needs_human_review": true
}
```

C. Safeguards

- **Structured prompts.** Instructions and untrusted data are separated; data are passed as JSON fields, not as free text mixed into instructions.
- **Output validation.** Responses are parsed against a JSON schema; enumerated fields must be from allowed values; every cited evidence field must exist in the input bundle with the same value; invalid outputs are discarded and the system falls back to a template-based explanation.
- **Restricted actions.** The LLM has no tool access. Its `recommended_action` is advisory. The policy engine computes the maximum permitted action from the risk band and threat type; the LLM cannot exceed it.
- **Sensitive-data minimization.** No request bodies, tokens, passwords, or full PII are sent. User IDs are pseudonymized; IPs are included only when needed for the explanation (or masked to a prefix). The LLM is accessed through a secure API with a retention/training policy chosen to fit the deployment's privacy needs [to be finalized].
- **Human approval.** High-impact actions (permanent block, endpoint restriction on critical routes, bulk revocation) need administrator confirmation.
- **Prompt-injection protection.** URL paths, query strings, headers (e.g., User-Agent), and any logged text are untrusted. We (i) send only extracted numeric/categorical features and length-limited, escaped strings, (ii) never place them in the instruction section, (iii) strip control characters and instruction-like patterns, (iv) validate outputs, and (v) test with an injection suite (Section XII). Because the LLM cannot execute actions, a successful injection is limited to degrading explanation text, which validation and human review can catch.
- **Determinism and cost control.** Low temperature, bounded output length, caching by incident fingerprint, and a fallback when the LLM service is unavailable so that detection and response never depend on it.

X. INTELLIGENT AUTOMATED INCIDENT RESPONSE

A. Risk score

For a detection window we compute

$R = 100 \cdot (w_1 \cdot p + w_2 \cdot a + w_3 \cdot s + w_4 \cdot f + w_5 \cdot e + w_6 \cdot h)$, with $\sum w_i = 1$ and each factor scaled to $[0, 1]$:

Symbol	Factor	Definition
p	ML prediction confidence	Calibrated probability of the predicted non-normal class
a	Anomaly score	Isolation Forest / One-Class SVM score, normalized by percentile against normal data
s	Threat severity	Lookup by threat class from a policy table (e.g., possible exfiltration > rate abuse)
f	Request frequency	Rate relative to a reference (e.g., $\min(1, \text{rate} / \text{rate_ref})$ or log-scaled)
e	Endpoint sensitivity	Configured level of the affected endpoints, scaled to $[0, 1]$

Symbol	Factor	Definition
h	Historical deviation	Sigmoid of the deviation from the entity's own baseline

Initial weights and thresholds are **design parameters, not results**. We will tune them on a validation set (e.g., logistic regression or grid search against a cost function that penalizes false blocks more than missed low-severity events). A cost-sensitive view (as in ALERT's cost-sensitive decisions [1]) can guide the objective. Deterministic **override rules** can raise the band regardless of R (e.g., successful access to another user's sensitive record; token used from two distant IPs simultaneously).

Risk bands (initial, tunable): Low < 30; Medium 30–59; High 60–84; Critical ≥ 85.

B. Response policy

Risk band	Default response	Notes
Low	Monitor	Log, tag entity, keep on watch list
Medium	Monitor and rate-limit	Per-session/IP token bucket; automatic expiry
High	Temporary IP block and/or session/token revocation	Revoke when the abuse is identity-bound (e.g., token misuse, enumeration by an authenticated user); block when source-bound (e.g., brute force); TTL-limited
Critical	Full block and administrator alert	Permanent or long-lived blocks and endpoint restriction need human approval

Endpoint restriction (temporarily limiting a sensitive endpoint to trusted roles or networks) and **administrator alert** are used as complements at High and Critical levels. All actions carry a TTL, an audit record, and a rollback path. Allow-lists protect health checks, internal services, and known partner IPs from accidental blocking. Cool-down and hysteresis prevent rapid oscillation between actions.

C. Control flow

1. Detector and classifier produce class, confidence, and anomaly score.
2. Risk engine computes R and band.
3. Policy engine derives the *permitted action set* and the *default action*.
4. LLM produces explanation and a recommended action within the permitted set (advisory).
5. Executor applies the default action (auto for Low–High with TTL; pending approval for Critical or high-impact actions). If the LLM recommends a stricter action than the default, it is presented to the analyst as an escalation suggestion, not executed automatically.
6. Outcome and analyst feedback are stored for retraining and weight tuning.

Automated actions are therefore policy-controlled. The LLM cannot invoke arbitrary security operations.

XI. IMPLEMENTATION

A. Technology stack

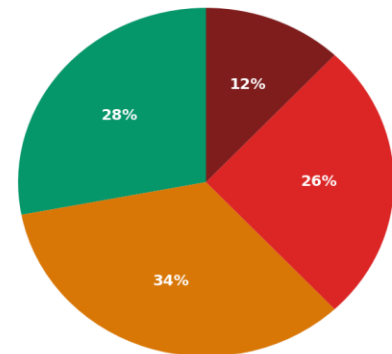
Layer	Technology	Planned use
Backend	Java, Spring Boot, REST APIs	Protected services; logging filter; incident, policy, and response services
Security	Spring Security, JWT / session management	Authentication, authorization, token revocation checks (e.g., a denylist keyed by token ID)
Rate limiting	A library such as Bucket4j, or a gateway feature [to be chosen]	Per-session/IP throttling
Frontend	React with Vite	Security dashboard
Database	PostgreSQL	API events, features, incidents, responses, audit log, policy configuration
ML	An ML service using a suitable library (e.g., scikit-learn/XGBoost in a Python microservice, or a Java-native library) exposed via REST [to be chosen]	Training and inference
LLM	An LLM accessed through a secure API/service [to be selected]	Explanation, summary, recommendation

We do not name a specific LLM, dataset, or performance figure here; these will be added when chosen and measured.

B. Main components

- **Logging filter:** a `OncePerRequestFilter`-style component capturing metadata and writing asynchronously.
- **Feature service:** scheduled or streaming aggregation of windows into feature vectors.
- **Detection client:** calls the ML service; stores class, confidence, and anomaly score.
- **Risk and policy services:** compute R and select actions from configuration.
- **LLM gateway:** builds the sanitized evidence bundle, calls the LLM, validates the response, and falls back to templates.
- **Response executor:** applies rate limits, IP deny entries with TTL, token revocation, endpoint restrictions, and notifications; enforces approval workflow.
- **Dashboard:** the panels listed in Section VII-J.

Fig. 9. Severity Distribution of Detected Threat Events (Illustrative Prototype Data)



C. Dashboard contents

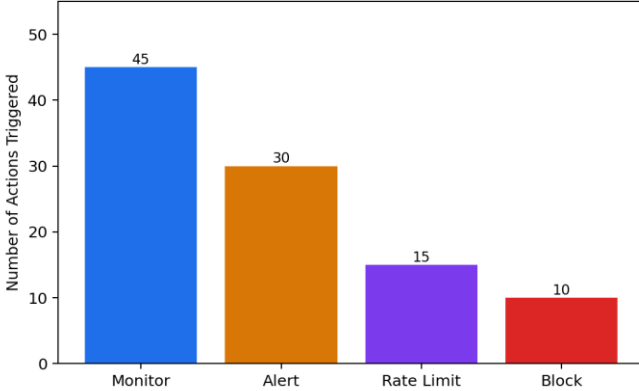
Total API requests; normal vs. suspicious vs. blocked requests; active incidents; threat categories; risk levels; recent incidents; source IP information; affected endpoints; LLM-generated explanations; and response actions taken (with approve/reject for pending actions).

D. Implementation status **(fill in)**

Component	Status (implemented / in progress / planned)	Notes
Logging filter and event storage	[]	
Feature extraction	[]	
Anomaly detection and classification	[]	
Risk scoring	[]	
Policy engine and response executor	[]	
LLM gateway and validation	[]	
Dashboard	[]	
Attack simulation scripts	[]	

This table lets the paper distinguish what was actually implemented from what remains future work, as required.

Fig. 10. Distribution of Automated Response Actions (Controlled Simulation)



XII. EXPERIMENTAL METHODOLOGY

This section describes the proposed evaluation. No results are claimed.

A. Research questions

- **RQ1:** Does API behavioral (window and sequence) detection outperform per-request detection on API-specific attacks?
- **RQ2:** Which model(s) give the best trade-off of detection, false positives, and latency?
- **RQ3:** Does graded, risk-based response reduce collateral impact compared with an ALLOW/MONITOR/BLOCK policy at similar detection performance?
- **RQ4:** Does the LLM layer produce faithful, useful explanations, and is it robust to prompt injection?
- **RQ5:** What is the runtime overhead of the full pipeline?

B. Test environment and data

1. **Testbed.** A target Spring Boot API (and optionally intentionally vulnerable applications from the community) with users, roles, and resources.
2. **Benign traffic.** Scripted and, where feasible, recorded realistic usage with varied user profiles, including legitimate bursts and bulk operations (to test false positives).
3. **Attack traffic.** Scripts for: brute force and credential stuffing; ID enumeration (sequential and shuffled); BOLA/IDOR cross-user access; rate abuse; token replay from multiple IPs; injection-style payloads; slow data exfiltration; abnormal call sequences. Attacks are run only against our own testbed.
4. **Optional public data.** FUR-API [13] and the Kaggle API access-behavior dataset [14] may be used if their schema supports our features; this must be checked.
5. **Splits.** Time-aware and scenario-aware splits (do not place the same attack script in both training and test), because random splits can produce optimistic results, a caveat also noted by BacAlarm's authors [2].

C. Baselines

ID	Baseline	Purpose
B1	Static rules/thresholds (e.g., N requests per minute)	Practical reference
B2	Per-request classifier (request-level features only)	Tests value of behavior
B3	ALERT-style baseline: log-level anomaly detection plus a cost-sensitive ALLOW/MONITOR/BLOCK policy, implemented from the published description	Reference to the base paper
B4	Ours without sequence/ID features	Ablation
B5	Ours without LLM layer	Ablation
B6	Full proposed system	

Caveat for B3: ALERT's code and full details may not be available; B3 is a *reasonable re-implementation of an ALERT-style approach*, not the original system, and must be described that way. We will not attribute B3's numbers to ALERT's authors.

D. Metrics

- **Detection:** accuracy, precision, recall, F1 (per class and macro), false-positive rate, attack detection rate; MCC and PR-AUC for class imbalance.
- **Timing:** detection latency (time from first malicious request to detection), response latency (detection to action applied).
- **Performance:** API throughput and p50/p95 latency with and without the monitoring filter (load tools such as k6 or JMeter); CPU/memory of each component.
- **Response quality:** rate of legitimate requests wrongly throttled or blocked; number of attack requests served before mitigation; appropriateness of action level relative to ground-truth severity.
- **Explanation quality:** a rubric (e.g., 1–5) for correctness, completeness, evidence faithfulness, and actionability rated by at least two evaluators with inter-rater agreement reported; automated faithfulness check (every cited

evidence value must match the input bundle); comparison with a template baseline.

- **Robustness:** injection test suite (malicious strings in paths, headers, and parameters) measuring whether explanations or recommendations are altered and whether validation catches it.

E. Statistical treatment

Repeated runs with different seeds; report means with confidence intervals; use appropriate significance tests for paired comparisons; report all hyperparameters and hardware.

F. Threats to validity

Synthetic or self-generated attacks may not represent real adversaries; the testbed is small; LLM outputs are non-deterministic; results may not generalize across applications; weights and thresholds tuned on validation data may overfit.

XIII. RESULTS AND DISCUSSION

Status: experiments not yet reported in this draft. The following templates are to be filled with measured values. Do not include numbers that were not produced by your own runs.

Table I: Detection performance (to be completed)

Model / Baseline	Precision	Recall	F1	FPR	MCC
B1 Rules	TBD	TBD	TBD	TBD	TBD
B2 Per-request	TBD	TBD	TBD	TBD	TBD
B3 ALERT-style	TBD	TBD	TBD	TBD	TBD
Isolation Forest	TBD	TBD	TBD	TBD	TBD
One-Class SVM	TBD	TBD	TBD	TBD	TBD
Logistic Regression	TBD	TBD	TBD	TBD	TBD
Random Forest	TBD	TBD	TBD	TBD	TBD
Gradient Boosting	TBD	TBD	TBD	TBD	TBD
XGBoost	TBD	TBD	TBD	TBD	TBD

Table II: Per-category validation status (to be completed)

Threat category	Attack scripts used	Validated?	Result
Authentication attacks	[]	[]	TBD
Brute force	[]	[]	TBD
Authorization violations	[]	[]	TBD
API enumeration	[]	[]	TBD
Injection-related	[]	[]	TBD
Rate abuse	[]	[]	TBD
Suspicious session	[]	[]	TBD
Abnormal sequences	[]	[]	TBD
Possible exfiltration	[]	[]	TBD

Table III: Latency and overhead (detection latency, response latency, throughput, p95 latency, CPU/memory) (TBD).

Table IV: Explanation quality and injection robustness (TBD).

Discussion guide. When results are available, discuss: (1) whether sequence/ID features improve recall on enumeration and BOLA-like scenarios (compare B2 vs. B6); (2) the false-positive/false-negative trade-off and which model was selected and why; (3) how graded responses changed collateral impact compared with B3; (4) whether the LLM explanations were faithful, and where they failed; (5) overhead and whether the pipeline is practical at the tested load; (6) limitations of the data. Report negative or mixed outcomes honestly.

XIV. SECURITY, PRIVACY, AND SAFETY CONSIDERATIONS

- **Privacy and minimization.** Store only metadata needed for detection; exclude bodies and credentials; pseudonymize identifiers; define retention limits; consider applicable regulations (e.g., GDPR/local law) when logging IPs and identities.
- **LLM data handling.** Send only minimized evidence; select a provider and configuration suitable for the data classification; keep logs of prompts and outputs for audit, with access control.
- **Prompt injection and untrusted content.** As in Section IX-C; the LLM has no execution authority.
- **Action safety.** TTLs, allow-lists, rate caps on automated actions (to prevent a mass-blocking incident), approval for high-impact actions, kill switch to disable automation, and full audit trail.
- **Self-denial-of-service risk.** Attackers may try to trigger blocks on legitimate IPs (e.g., shared NATs); IP-only blocks are time-limited and prefer session/token controls where identity is known.
- **Model attacks.** Attackers may slow-roll traffic to evade detection or poison training data; we propose periodic retraining with reviewed data and monitoring for drift.
- **System hardening.** Secure the ML and LLM service endpoints, secrets, and dashboard access (role-based access for analysts).
- **Ethical testing.** Attack simulations are executed only on systems owned by the team.

XV. LIMITATIONS AND FUTURE WORK

Limitations.

- Empirical validation is pending; category coverage is a design goal.
- Reliance on labeled or scripted attack data; real-world traffic may differ.
- Thresholds and weights require per-application tuning; endpoint sensitivity is manually configured.
- LLM output can be wrong or inconsistent; validation reduces but does not eliminate risk.
- Our description of ALERT is based on its abstract.
- CVE intelligence, which ALERT includes, is not part of our current scope.

Future work.

- Integrate CVE/dependency intelligence to raise risk for endpoints with known vulnerable components.
- Add invariant or schema-based checks, as in MINES [3], and LLM-simulated traffic for training, as in BacAlarm [2], to reduce reliance on scripted attacks.
- Add online learning from analyst feedback and adaptive policy tuning, in the spirit of ALERT's online adaptation [1].

- Support gateway and service-mesh deployments and multi-tenant operation.
- Extend to GraphQL/gRPC APIs and to LLM-facing endpoints.
- Conduct a systematic literature review and a user study with analysts.

XVI. CONCLUSION

This paper proposed an API-focused extension of the closed-loop SOC agent concept introduced by ALERT. We acknowledged ALERT's existing use of LLM assistance and automated response, and centered our contribution on two API-specific improvements: behavior-aware detection across request sequences, and a risk-driven graded response ladder governed by a policy engine. The LLM is deliberately limited to contextual reasoning, explanation, summarization, and recommendation, with safeguards against unsafe actions, data leakage, and prompt injection. We presented the architecture, features, risk model, implementation plan, and a rigorous evaluation methodology including ALERT-style baselines and ablations. The next step is to complete implementation and report measured results; only then can the practical benefits of the proposed extensions be assessed.

REFERENCES

- [1] T. Sooksomstarn, Q. Wu, and A. Kanai, "ALERT: A closed-loop SOC agent for web and API security with CVE-based intelligence and adaptive anomaly detection," in *Proc. 2026 IEEE Int. Conf. Consumer Electronics (ICCE)*, 2026, doi: 10.1109/ICCE67443.2026.11449910.
- [2] Y. Yang, H. Zhang, B. Liu, J. Xu, J. Hu, L. Dong, Z. Mao, and D. Pan, "BacAlarm: Mining and simulating composite API traffic to prevent broken access control violations," arXiv:2512.19997, Dec. 2025.
- [3] W. Zhang, Y. Lin, C. F. A. Kwok, X. Teoh, X. Xie, F. Liauw, H. Zhang, and J. S. Dong, "MINES: Explainable anomaly detection through web API invariant inference," in *Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE '26)*, Rio de Janeiro, Brazil, 2026, doi: 10.1145/3744916.3773160 (arXiv:2512.06906).
- [4] Z. Dong, Y. Huang, S. Sun, W. Cui, and Z. Wang, "BLADE: Behavior-level anomaly detection using network traffic in web services," arXiv:2511.05193, Nov. 2025.
- [5] E. Eilertsen, V. Mavroeidis, and G. Grov, "Towards agentic investigation of security alerts," in *Proc. 2025 IEEE Int. Conf. Big Data*, pp. 7793–7802; arXiv:2604.25846. [Verify the venue/date attribution against the source.]
- [6] "DAST-AI: Reducing false positives in API security testing via LLM-based semantic analysis," *SBSEg*, 2026. [Add authors and exact venue details from <https://sol.sbc.org.br/index.php/sbseg/article/view/44304>.]
- [7] "Broken Object Level Authorization in the wild: An empirical taxonomy from 100+ bug bounty disclosures," 2026. [Add authors and identifier; the source we accessed was a Hugging Face Daily Papers listing.]
- [8] "A survey on agentic security: Applications, threats and defenses," arXiv:2510.06445, 2025. [Add authors.] (Used only as a secondary source for the works of Turcotte et al. and Tellache et al., 2025.)
- [9] "Unlocking deeper understanding: Leveraging explainable AI for API anomaly detection insights," *ACM*, 2024, doi: 10.1145/3651671.3651738. [Add authors and proceedings title.]
- [10] "HuntGPT: Integrating machine learning-based anomaly detection and explainable AI with large language models (LLMs)," arXiv:2309.16021, 2023. [Add authors.]
- [11] OWASP Foundation, "OWASP API Security Top 10 – 2023." [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x00-header/>
- [12] OWASP Foundation, "OWASP Top 10 for LLM Applications 2025." [Online]. [Add URL from the OWASP site.]
- [13] Y. Liu, H. Yu, F. Dai, X. Gu, C. Cui, B. Li, and W. Wang, "FURAPI: Dataset and baselines toward realistic API anomaly detection," in *Proc. IEEE ICASSP*, 2024, pp. 4525–4529.
- [14] R. Guntur, "API security: Access behavior anomaly dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/tangodelta/api-access-behaviour-anomaly-dataset> [Check license and schema before use.]
- [15] U. Aharon, R. Dubin, A. Dvir, and C. Hajaj, "A classification-by-retrieval framework for few-shot anomaly detection to detect API injection," *Computers & Security*, vol. 150, art. 104249, 2025.

Reference hygiene (delete before submission). I confirmed the existence, titles, and abstract-level content of [1]–[10], [13], [15] through web sources, but several entries above lack author lists or venue details that I could not confirm; these are flagged. Please open each source, complete the metadata, and remove any reference you do not actually read. [11], [12], and [14] are cited as resources and should also be checked. Reference [12] is currently cited nowhere in the text; either cite it in Section IX-C (LLM risks) or remove it.